

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-aba6c425d56dd0b9b.jsonl`  
- SHA-256: `251f81c5211d00c34dd999bd436d58c96ee02875c521c74cf466f3fa1cca0adf`  
- Source modified: `2026-06-25T17:01:13+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_c81dbe7a-7ff`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T16:57:30.533Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [NIT] `_file_mock` sets `state/state.name` that the code under test never reads
Location: `tests/test_provider_gemini.py:109-116`
Detail: ``cleanup_orphaned_files`` only reads ``mime_type``, ``create_time``, and ``name``. The ``state`/`state.name`` attributes set by the helper are dead scaffolding for these tests ? harmless but slightly misleading (suggests state filtering is part of the contract when it is not). Pagination is also only single-page (``iter([...])``); since prod relies on the SDK Pager's own multi-page iteration and has no manual pagination code, this is acceptable, but worth a note.
Suggested fix: Drop the ``state`` parameter from ``_file_mock`` (or add a brief comment that the sweep intentionally ignores `File.state`). No action needed on pagination since there is no prod pagination logic to test.

Return `isReal` + `adjustedSeverity` (`INVALID` if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:57:32.291Z)

I'll verify this finding by examining the actual code in the PR.

3. user (2026-06-25T16:57:34.047Z)

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
o1-p11/gemini-remote-file-gc

4. user (2026-06-25T16:57:34.617Z)

```
backend/main.py          | 25 ++++++++  
backend/providers/gemini.py | 43 ++++++++  
tests/test_provider_gemini.py | 104 ++++++++  
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:57:36.444Z)

Let me read the test file and the production code to verify the finding.

6. user (2026-06-25T17:00:45.494Z)

```
1      import logging
```

```

2     from unittest.mock import MagicMock, patch
3
4     from backend.providers.gemini import GeminiProvider
5
6
7     def test_gemini_provider_generate_failure_is_flagged():
8         """A generate() give-up (per-attempt timeout / 503 burst after retries) must set
9         metrics['generate_failed'] so a FAILED chunk isn't silently mistaken for 'no
rallies'
10        (the silent-partial-result guard the segmenter surfaces loudly)."""
11        with patch("backend.providers.gemini.genai") as mock_genai, \
12            patch("backend.providers.gemini.run_bounded", side_effect=RuntimeError("503
exhausted")):
13            client_mock = MagicMock()
14            mock_genai.Client.return_value = client_mock
15            file_mock = MagicMock()
16            file_mock.state.name = "ACTIVE"
17            file_mock.name = "dummy_file"
18            client_mock.files.upload.return_value = file_mock
19            client_mock.files.get.return_value = file_mock
20
21            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
22            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
23
24            assert segments == [] # no analysis for this chunk
25            assert metrics.get("generate_failed") is True # ...but the failure is FLAGGED,
not silent
26
27
28    def test_gemini_provider_success():
29        # Mock GenAI client
30        with patch("backend.providers.gemini.genai") as mock_genai:
31            client_mock = MagicMock()
32            mock_genai.Client.return_value = client_mock
33
34            # Mock file upload
35            file_mock = MagicMock()
36            file_mock.uri = "gs://dummy"
37            file_mock.name = "dummy_file"
38            file_mock.state.name = "ACTIVE"
39
40            client_mock.files.upload.return_value = file_mock
41            client_mock.files.get.return_value = file_mock
42
43            # Mock generate_content response
44            response_mock = MagicMock()
45            response_mock.text = '[{"start_time": 1.0, "end_time": 2.0}]'
46            client_mock.models.generate_content.return_value = response_mock
47
48            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
49            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
50
51            assert len(segments) == 1
52            assert segments[0]["start_time"] == 1.0
53            assert metrics["upload_time"] >= 0
54            assert metrics["process_time"] >= 0
55            assert metrics["gen_time"] >= 0
56
57            # Ensure delete is called
58            client_mock.files.delete.assert_called_with(name="dummy_file")
59

```

```

60     def test_gemini_provider_invalid_json():
61         with patch("backend.providers.gemini.genai") as mock_genai:
62             client_mock = MagicMock()
63             mock_genai.Client.return_value = client_mock
64             file_mock = MagicMock()
65             file_mock.state.name = "ACTIVE"
66             client_mock.files.upload.return_value = file_mock
67             client_mock.files.get.return_value = file_mock
68
69             response_mock = MagicMock()
70             response_mock.text = '{"not_a_list": true}'
71             client_mock.models.generate_content.return_value = response_mock
72
73             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
74             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
75             assert len(segments) == 0
76
77     def test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
78         with patch("backend.providers.gemini.genai") as mock_genai:
79             client_mock = MagicMock()
80             mock_genai.Client.return_value = client_mock
81             # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
82             monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
83                                 lambda p: 600 * 1024 * 1024)
84             provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
85             with caplog.at_level(logging.ERROR, logger="backend.providers.gemini"):
86                 segments, metrics = provider.analyze_video("big.mp4", "p")
87             assert segments == []
88             client_mock.files.upload.assert_not_called()
89             # The refusal must be LOUD and machine-visible: callers aggregate the metrics
90             # flag into the per-video report (oversize_clip_skipped rule).
91             assert metrics["oversize_skipped"] is True
92             assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
93
94
95     def test_gemini_provider_upload_fail():
96         with patch("backend.providers.gemini.genai") as mock_genai:
97             client_mock = MagicMock()
98             mock_genai.Client.return_value = client_mock
99             client_mock.files.upload.side_effect = Exception("Upload err")
100
101             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
102             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
103             assert len(segments) == 0
104
105
106
107     # --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone

```

```

122     stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123     fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
124     files = [
125         _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
126         _file_mock("fresh-vid", "video/quicktime", fresh),  # fresh video -> kept (grace)
127         _file_mock("old-img", "image/png", stale),          # non-video -> kept
128         _file_mock("notts", "video/mp4", None),            # no timestamp -> kept (safe)
129     ]
130     with patch("backend.providers.gemini.genai"):
131         provider = GeminiProvider(api_key="k", model_name="m")
132         provider._client = MagicMock()
133         provider._client.files.list.return_value = iter(files)
134         deleted = provider.cleanup_orphaned_files(grace_sec=3600)
135     assert deleted == 1
136     provider._client.files.delete.assert_called_once_with(name="old-vid")
137
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
141         with patch("backend.providers.gemini.genai"):
142             provider = GeminiProvider(api_key="k", model_name="m")
143             provider._client = MagicMock()
144             provider._client.files.list.side_effect = RuntimeError("network")
145             assert provider.cleanup_orphaned_files() == 0
146
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),
154             _file_mock("b", "video/mp4", stale),
155         ]
156         with patch("backend.providers.gemini.genai"):
157             provider = GeminiProvider(api_key="k", model_name="m")
158             provider._client = MagicMock()
159             provider._client.files.list.return_value = iter(files)
160             # First delete raises, second succeeds ? _safe_delete swallows it.
161             provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163             assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()

```

```

186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
197         startup)."""
198         import backend.main as main_mod
199         cfg = MagicMock()
200         cfg.ai_provider = "gemini"
201         cfg.ai_model = "gemini-2.5-flash"
202         provider = MagicMock()
203         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
204         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
205         monkeypatch.setenv("GEMINI_API_KEY", "k")
206         # Must not raise.
207         main_mod._maybe_sweep_gemini_orphans(cfg)
208         provider.cleanup_orphaned_files.assert_called_once()

```

7. user (2026-06-25T17:00:45.495Z)

```

1     import json
2     import logging
3     import os
4     import time
5     from datetime import datetime, timedelta, timezone
6     from typing import Tuple
7
8     from google import genai
9     from google.genai import types
10
11     from backend.infrastructure.execution_policy import (
12         ExecutionPolicy,
13         PolicyTimeout,
14         poll_until,
15         run_bounded,
16     )
17     from backend.providers.base import AIVideoProvider
18
19     logger = logging.getLogger(__name__)
20
21     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22     # each uploaded clip is below this. Oversize uploads are refused (not silently
23     attempted).
24     MAX_UPLOAD_MB = 500
25
26     class GeminiProvider(AIVideoProvider):
27         def __init__(self, api_key: str, model_name: str):
28             super().__init__(api_key, model_name)
29             # Initialize client lazily to avoid connection/auth issues during class
30             instantiation
31             self._client = None
32
33     @property
34     def client(self):
35         if self._client is None:
36             self._client = genai.Client(api_key=self.api_key)
37         return self._client

```

```

37 def _safe_delete(self, video_file) -> None:
38     """Best-effort cleanup of an uploaded File (never raises)."""
39     try:
40         self.client.files.delete(name=video_file.name)
41     except Exception:
42         pass
43
44 def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45     """Sweep and delete video files orphaned on the Gemini File API by a previous
46     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48     that reach a ``finally``/except).
49
50     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
51     whose
52     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
53     upload).
54     Never raises; returns the count deleted. Intended to run once at startup.
55
56     Args:
57         grace_sec: only delete files older than this (default 1h). Protects a file a
58         live worker may still be processing.
59     """
60     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
61     deleted = 0
62     try:
63         files = list(self.client.files.list())
64     except Exception as e:
65         logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
66         %s", e)
67         return 0
68     for f in files:
69         try:
70             mime = getattr(f, "mime_type", "") or ""
71             if not mime.startswith("video/"):
72                 continue
73             # f.create_time is an RFC3339 string on the File API.
74             ct = getattr(f, "create_time", None)
75             if ct is None:
76                 # No timestamp ? can't prove it's stale; leave it (safe default).
77                 continue
78             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
79             if created > cutoff:
80                 continue # too recent ? a live worker may own it
81             self.client.files.delete(name=f.name)
82             deleted += 1
83         except Exception as e:
84             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
85     if deleted:
86         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
87         deleted, grace_sec)
88     return deleted
89
90 def analyze_video(self, video_path: str, prompt: str,
91     chunk_duration: float = 0.0,
92     label: str = "") -> Tuple[list, dict]:
93     log_prefix = f"Gemini {label}" if label else "Gemini"
94
95     metrics: dict = {
96         "upload_time": 0.0,
97         "process_time": 0.0,
98         "gen_time": 0.0,
99     }
100
101     # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for

```

```

safety/speed)
98         try:
99             size_mb = os.path.getsize(video_path) / (1024 * 1024)
100         except OSError:
101             size_mb = 0.0
102         if size_mb > MAX_UPLOAD_MB:
103             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                         f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                         f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106             metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107             metrics["oversize_skipped"] = True
108             return [], metrics
109
110         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111         logger.info(f"[{log_prefix}] Uploading {video_path}...")
112         t_upload_start = time.time()
113         video_file = None
114         for attempt in range(3):
115             try:
116                 video_file = self.client.files.upload(file=video_path)
117                 metrics["upload_time"] = time.time() - t_upload_start
118                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                 break
120             except Exception as e:
121                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                 if attempt < 2:
123                     time.sleep(2 * (attempt + 1))
124         if video_file is None:
125             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126             metrics["error"] = "upload failed after 3 attempts"
127             return [], metrics
128
129         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130         # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131         t_process_start = time.time()
132         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134         def _processed():
135             nonlocal video_file
136             video_file = self.client.files.get(name=video_file.name)
137             return None if video_file.state.name == "PROCESSING" else video_file
138
139         try:
140             video_file = poll_until(_processed, process_policy,
label=f"[{log_prefix}] processing", logger=logger)
141         except PolicyTimeout:
142             logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
143             self._safe_delete(video_file)
144             metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
145             return [], metrics
146         except Exception as e:
147             logger.error(f"[{log_prefix}] Status check failed: {e}")
148             self._safe_delete(video_file)
149             metrics["error"] = f"status check failed: {e}"
150             return [], metrics
151

```

```

152         if video_file.state.name == "FAILED":
153             logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154             self._safe_delete(video_file)
155             metrics["error"] = f"video processing failed: {video_file.error.message}"
156             return [], metrics
157
158         metrics["process_time"] = time.time() - t_process_start
159         logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
160
161         # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
162         # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
163         # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165         t_gen_start = time.time()
166         gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167         raw_text = None
168
169         def _generate():
170             return self.client.models.generate_content(
171                 model=self.model_name,
172                 contents=[video_file, prompt],
173                 config=types.GenerateContentConfig(
174                     response_mime_type="application/json", temperature=0.0),
175             )
176
177         try:
178             response = run_bounded(_generate, gen_policy, label=f"[{log_prefix}]
generate", logger=logger)
179             raw_text = response.text
180             metrics["gen_time"] = time.time() - t_gen_start
181             logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182             except Exception as e:                                # PolicyTimeout (hang) or the last 503 ?
both end here
183                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184                 metrics["error"] = f"generate failed after retries: {e}"
185                 metrics["generate_failed"] = True
186                 self._safe_delete(video_file)
187
188             if not raw_text:
189                 return [], metrics
190
191         # 4. Parse JSON
192         try:
193             segments = json.loads(raw_text)
194             if not isinstance(segments, list):
195                 logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                 metrics["error"] = "response is not a JSON array"
197                 return [], metrics
198             return segments, metrics
199         except Exception as e:
200             logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201             metrics["error"] = f"JSON parse failed: {e}"
202             return [], metrics
203

```


I have read both files. Let me verify the finding's claims against the actual code.

The ``cleanup_orphaned_files`` method (lines 44-84) reads exactly these attributes from each file ``f``:

- ``f.mime_type`` (line 67, via ``getattr``)
- ``f.create_time`` (line 71, via ``getattr``)
- ``f.name`` (line 78)

It never reads ``f.state`` or ``f.state.name``. The ``_file_mock`` helper (lines 109-116) sets ``f.state = MagicMock()`` and ``f.state.name = state``. So the finding's core claim is accurate: the ``state`/`state.name`` attributes set by the helper are not read by ``cleanup_orphaned_files``.

Let me check whether ``state`` is read anywhere else that the helper-built mocks flow into, and confirm the pagination claim.

9. user (2026-06-25T17:00:52.328Z)

```
137:         return None if video_file.state.name == "PROCESSING" else video_file
152:         if video_file.state.name == "FAILED":
```

10. user (2026-06-25T17:00:52.796Z)

```
61:         files = list(self.client.files.list())
```

11. assistant (2026-06-25T17:01:01.187Z)

The two ``state.name`` reads in production (lines 137, 152) are in ``analyze_video``, which uses a different ``file_mock`` (the top-of-file ones at lines 16, 38, 65 that legitimately set ``state.name``). Those tests genuinely exercise the state-based polling/failure logic, so setting ``state.name`` there is correct and load-bearing.

The ``_file_mock`` helper at lines 109-116 is used ONLY by the three ``cleanup_orphaned_files`` tests (lines 119, 148, 152 callers at 126-128, 152-155). The ``cleanup_orphaned_files`` sweep never inspects ``state`` ? it filters solely on ``mime_type`` + ``create_time``, then deletes by ``name``. So the ``state`/`state.name`` lines in ``_file_mock`` are indeed dead scaffolding for those tests.

Pagination claim: production uses ``list(self.client.files.list())`` (line 61), relying on the SDK's own iterator/pager. The tests stub it with ``iter([...])``, single-page. There's no manual pagination code in prod, so single-page mocks are an accurate test of what exists. The finding correctly characterizes this as acceptable.

This is correctly a NIT, not a bug: the dead ``state`` attributes are harmless (MagicMock would auto-create them on access anyway), cause no test failure or coverage gap, and only slightly mislead a reader into thinking state-filtering is part of the GC contract. The finding's facts are all verified against the actual code, and the severity (cosmetic/clarity, no functional impact) matches NIT.

12. user (2026-06-25T17:01:07.529Z)

Structured output provided successfully

13. assistant (2026-06-25T17:01:13.563Z)

I've completed the verification. The finding is real and correctly classified as a NIT.

Summary: The finding accurately describes the code. In ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\providers\gemini.py``, ``cleanup_orphaned_files`` (lines 44-84) filters only on ``mime_type`` and ``create_time``, deleting by ``name`` ? it never reads ``f.state`/`f.state.name``. The ``_file_mock`` helper at ``tests/test_provider_gemini.py:109-116`` is used only by the three GC tests, so its ``state`/`state.name`` lines are dead scaffolding. The production ``state.name`` reads (gemini.py:137, 152) belong to ``analyze_video``, which is covered by the separate top-of-file mocks, not by ``_file_mock``. The pagination note is also correct: prod relies on the SDK pager with no manual paging, so the single-page ``iter([...])`` stubs are

appropriate. Purely a clarity nit with no functional or coverage impact.